

A Report on Mini Project

**DATA-DRIVEN FUEL CONSUMPTION PREDICTION THROUGH
MACHINE LEARNING**

Submitted to the

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD

In partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

BY

J VARSHA RAO (21WJ1A05C6)

K NIKHIL SAGAR (21WJ1A05D3)

Under the Guidance of

Mr. Arun Singh Kaurav

(Assistant Professor, Dept. of CSE)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

GURU NANAK INSTITUTIONS TECHNICAL CAMPUS (AUTONOMOUS)

SCHOOL OF ENGINEERING & TECHNOLOGY

(Approved by AICTE, New Delhi, permanently affiliated to JNTUH

An Autonomous, NBA, NAAC A+ Accredited Institution)

Ibrahimpattanam, Hyderabad – 501506, Telangana

2024 – 2025



GURU NANAK INSTITUTIONS TECHNICAL CAMPUS



AUTONOMOUS
under Section
2(f) & 12(b) of
University Grants
Commission Act

Date: 4-1-2025

Department of Computer Science & Engineering

CERTIFICATE

This is to Certify that this Mini project report entitled **“DATA-DRIVEN FUEL CONSUMPTION PREDICTION THROUGH MACHINE LEARNING”** by **J VARSHA RAO (21WJ1A05C6), K NIKHIL SAGAR (21WJ1A05D3)** was submitted in partial fulfilment of the requirements for the degree of **Bachelor of Technology in Computer Science & Engineering** of the **Guru Nanak Institutions Technical Campus (Autonomous)** during the academic year 2024-2025, is a Bonafide record of work carried out under our guidance and supervision.

INTERNAL GUIDE

Mr. Arun Singh Kaurav

Assistant Professor, CSE

PROJECT CO-ORDINATOR

Mr. Md. Irfan

Assistant Professor, CSE

HEAD OF THE DEPARTMENT

Dr. Geeta Tripathi

EXTERNAL EXAMINER



City Office: B2, 2nd Flr, Above Bata, Vikrampuri Colony, Karkhana Road, Secunderabad – 500 009, Telangana, India.
Ph: +91-40-6632 3294, 6517 6117, Fax: +91-40-2789 2633

Campus: Ibrahimpatnam, R. R. District, Hyderabad – 501 506, Telangana, India. Ph: (0/95) 8414-20 21 20/ 21.



RAM Innovative Infotech

M : +91 9581 012 012

E : raminnovativeinfotech@gmail.com

Flat No.#309, Amrutha Ville,
Opp: Yashoda Hospital, Somajiguda,

Hyderabad-82, Telangana, India

www.raminnovativeinfotech.webs.com

PROJECT COMPLETION CERTIFICATE

This is to certify that the following students of final year B.Tech Department **Computer Science and Engineering** -Guru Nanak Institutions Technical Campus (GNITC) have completed their training and project at GNITC successfully.

STUDENT NAME:

ROLL NO:

1. **J Varsha Rao**

21WJ1A05C6

2. **K Nikhil Sagar**

21WJ1A05D3

The training was conducted on **ML-DL with Python** Technology for the Completion of the project title "**DATA-DRIVEN FUEL CONSUMPTION PREDICTION THROUGH MACHINE LEARNING**" in **October 2024**. This project has been completed in all aspects.



ACKNOWLEDGEMENT

We want to express our sincere gratitude to our guide **Mr. Arun Singh Kaurav, Assistant Professor**, Department of Computer Science & Engineering, for his excellent guidance and invaluable support, which helped us to accomplish our Bachelor's degree and prepared us to achieve more life goals in the future. His total support of our dissertation and countless technical and professional development contributions made for a delightful and fruitful experience.

We are very much grateful to our **Project Coordinator, Mr. Mohd. Irfan, Assistant Professor** of CSE, GNITC, Hyderabad, who is not only shown utmost patience but was fertile in suggestions, vigilant in directions of error, and have been infinitely helpful.

We are also thankful to **Dr. Geeta Tripathi, Professor & HOD CSE-1**, GNITC, Hyderabad, for being so helpful with valuable insights and guidance during our project.

We wish to express our sincere thanks to **Dr. Rishi Sayal, Professor & Associate Director** of CSE , GNITC for providing us with the conducive environment for carrying through our academic schedules and project with ease.

On a more personal note, we thank our beloved parents and friends for their moral support during our project.

J VARSHA RAO (21WJ1A05C6)

K NIKHIL SAGAR (21WJ1A05D3)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	LIST OF FIGURES	V
	LIST OF SYMBOLS	VI-VIII
	ABBREVIATIONS	IX
	ABSTRACT	X
1.	CHAPTER 1: INTRODUCTION	
	1.1 GENERAL	1
	1.2 SCOPE OF THE PROJECT	2
	1.3 OBJECTIVE	2
	1.3.1 EXISTING SYSTEM DISADVANTAGES	3
2.	CHAPTER 2: LITERATURE REVIEW	
	2.1 LITERATURE SURVEY	4-7
	2.2 PROPOSED SYSTEM	7
	2.2.1 PROPOSED SYSTEM ADVANTAGES	8
3.	CHAPTER 3: PROJECT DESCRIPTION	
	3.1 GENERAL	9
	3.2 METHODOLOGIES	9
	3.2.1 MODULES NAME	9
	3.2.2 MODULES EXPLANATION	9-10
	3.3 TECHNIQUE OR ALGORITHM	10-11
	3.3.1 EXISTING TECHNIQUE	10
	3.3.2 PROPOSED TECHNIQUE	11
4.	CHAPTER 4: REQUIREMENTS	
	4.1 GENERAL	12
	4.2 HARDWARE REQUIREMENTS	12
	4.3 SOFTWARE REQUIREMENTS	13
	4.4 FUNCTIONAL REQUIREMENTS	13-14

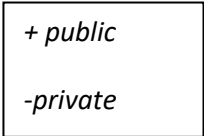
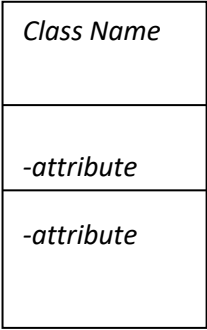
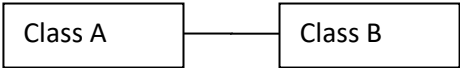
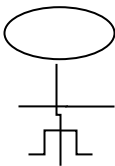
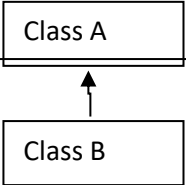
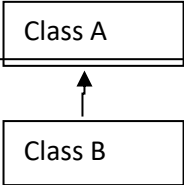
5.	CHAPTER 5: SYSTEM DESIGN 5.1 GENERAL 5.2 UML DIAGRAMS 5.2.1 USE CASE DIAGRAM 5.2.2 CLASS DIAGRAM 5.2.3 OBJECT DIAGRAM 5.2.4 STATE DIAGRAM 5.2.5 ACTIVITY DIAGRAM 5.2.6 SEQUENCE DIAGRAM 5.2.7 COLLABORATION DIAGRAM 5.2.8 COMPONENT DIAGRAM 5.2.9 DEPLOYMENT DIAGRAM 5.2.10 SYSTEM ARCHITECTURE	15 15-21 15-16 16 16-17 17 18 19 19-20 20 21 21
6.	CHAPTER 6: DEVELOPMENT TOOLS 6.1 1 PYTHON 6.2 HISTORY OF PYTHON 6.3 IMPORTANCE OF PYTHON 6.4 FEATURES OF PYTHON 6.5 LIBRARIES USED IN PYTHON	22 22 22-23 23-24 24
7.	CHAPTER 7: IMPLEMENTATION AND SNAPSHOTS 7.1 IMPLEMENTATION 7.2 VARIOUS SNAPSHOTS	25-34 35-36
8.	CHAPTER 8: SOFTWARE TESTING 8.1 GENERAL 8.2 DEVELOPING METHODOLOGIES 8.3 TYPES OF TESTING 8.3.1 UNIT TESTING 8.3.2 FUNCTIONAL TESTING 8.3.3 SYSTEM TESTING 8.3.4 PERFORMANCE TESTING	37 37 37-39 37 38 38 38

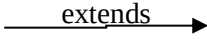
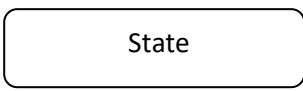
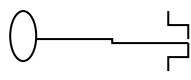
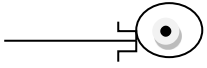
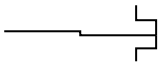
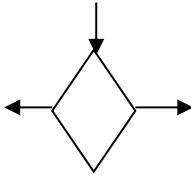
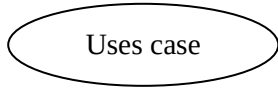
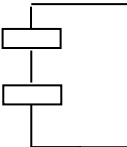
	8.3.5 INTEGRATION TESTING	38
	8.3.6 ACCEPTANCE TESTING	39
	8.3.7 BUILD THE TEST PLAN	39
9.	CHAPTER 9: FUTURE ENHANCEMENT 9.1 FUTURE ENHANCEMENT	40
10	CHAPTER 10: CONCLUSION 10.1 CONCLUSION	41
	REERENCES	42-43
	BASE PAPER	44-58

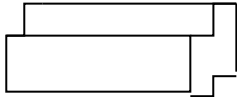
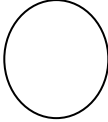
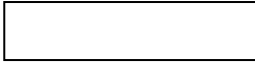
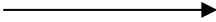
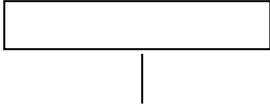
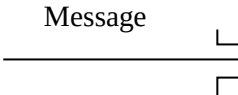
LIST OF FIGURES

FIGURE NO	NAME OF THE FIGURE	PAGE NO.
5.2.1	Use case Diagram	16
5.2.2	Class Diagram	17
5.2.3	Object Diagram	18
5.2.4	State Diagram	19
5.2.5	Activity Diagram	20
5.2.6	Sequence Diagram	21
5.2.7	Collaboration Diagram	22
5.2.8	Component Diagram	23
5.2.9	Deployment Diagram	24
5.2.10	System Architecture	25

LIST OF SYMBOLS

S.NO	NOTATION NAME	NOTATION	DESCRIPTION
1.	Class	 	Represents a collection of similar entities grouped together.
2.	Association	 	Associations represents static relationships between classes. Roles represents the way the two classes see each other.
3.	Actor		It aggregates several classes into a single classes.
4.	Aggregation	 	Interaction between the system and external environment

5.	Relation (uses)	uses	Used for additional process communication.
6.	Relation (extends)		Extends relationship is used when one use case is similar to another use case but does a bit more.
7.	Communication		Communication between various use cases.
8.	State		State of the processes.
9.	Initial State		Initial state of the object
10.	Final state		Final state of the object
11.	Control flow		Represents various control flow between the states.
12.	Decision box		Represents decision making process from a constraint
13.	Use case		Interact ion between the system and external environment.
14.	Component		Represents physical modules which are a collection of components.

15.	Node		Represents physical modules which are a collection of components.
16.	Data Process/State		A circle in DFD represents a state or process which has been triggered due to some event or action.
17.	External entity		Represents external entities such as keyboard, sensors, etc.
18.	Transition		Represents communication that occurs between processes.
19.	Object Lifeline		Represents the vertical dimensions that the object communications.
20.	Message		Represents the message exchanged.

LIST OF ABBREVIATIONS

S.NO	ABBREVIATION	EXPANSION
1.	DB	Database
2.	SVM	Support Vector Machine
3.	RF	Random Forest
4.	OBD	On-Board Diagnostics
5.	RPM	Revolutions Per Minute

ABSTRACT

In this project, we develop a machine learning model to predict vehicle fuel consumption using the Logistic Regression algorithm. The aim is to enhance the accuracy of fuel consumption predictions to achieve better fuel efficiency, leading to economic benefits and meeting industry needs. The proposed model considers key features such as Mass Air Flow, Vehicle Speed, Revolutions Per Minute, and Throttle Position Sensor to estimate fuel consumption. Our model is trained and tested on a comprehensive On-Board Diagnostics dataset that includes 18 features. By leveraging Logistic Regression, we provide a straightforward, interpretable, and efficient approach to predicting fuel consumption. The results demonstrate the model's capability to deliver reliable predictions, offering an alternative to more complex algorithms. This project underscores the potential of Logistic Regression in creating practical and effective fuel consumption prediction models, aiding manufacturers in developing strategies to minimize fuel usage and improve vehicle performance.

KEYWORDS:

- SVM
- RANDOM FOREST
- REGRESSION MODEL
- MACHINE LEARNING
- OBD
- RPM
- ARTIFICIAL NEURAL NETWORK

CHAPTER-1

INTRODUCTION

1.1 GENERAL:

Fuel consumption models for vehicles are of interest to manufacturers, regulators, and consumers. They are needed across all the phases of the vehicle life-cycle. In this paper, we focus on modeling average fuel consumption for heavy vehicles during the operation and maintenance phase. In general, techniques used to develop models for fuel consumption fall under three main categories:

- Physics-based models, which are derived from an in depth understanding of the physical system. These models describe the dynamics of the components of the vehicle at each time step using detailed mathematical equations [1], [2].
- Machine learning models, which are data-driven and represent an abstract mapping from an input space consisting of a selected set of predictors to an output space that represents the target output, in this case average fuel consumption [3], [4].
- Statistical models, which are also data-driven and establish a mapping between the probability distribution of a selected set of predictors and the target outcome [5], [6]. Trade-offs among the above techniques are primarily with respect to cost and accuracy as per the requirements of the intended application.

The prediction of vehicle fuel consumption is crucial for optimizing operational costs and environmental impact in various industries, including automotive manufacturing, logistics, and transportation. By accurately estimating fuel usage, businesses can implement strategies to improve fuel efficiency, reduce carbon emissions, and enhance overall operational performance. Machine learning techniques have emerged as powerful tools in tackling the complexity of fuel consumption prediction. These techniques leverage large datasets containing diverse parameters such as engine characteristics, driving conditions, and environmental factors to model and predict fuel consumption patterns effectively. Among these techniques, logistic regression stands out for its simplicity, interpretability, and capability to handle binary classification tasks, making it suitable for predicting fuel consumption based on key input features.

In this context, this study proposes the development of a machine learning-based fuel consumption prediction model. The model will utilize logistic regression to analyze and correlate factors such as Mass Air Flow, Vehicle Speed, Revolutions Per Minute (RPM), and Throttle Position Sensor (TPS) readings. These variables are critical in determining the amount of fuel consumed during vehicle operation. The model will be trained and evaluated using a comprehensive dataset obtained from On-Board Diagnostics (OBD) systems, which capture real-time vehicle performance metrics. By harnessing the predictive capabilities of logistic regression, this research aims to provide accurate estimations of fuel consumption under varying driving conditions and operational scenarios. The outcomes of this study are expected to contribute to the development of practical tools for fuel efficiency optimization and environmental sustainability in the automotive sector. By enhancing our understanding of fuel consumption patterns through machine learning, this research seeks to empower stakeholders with actionable insights for informed decision-making and proactive management of vehicle fleets.

1.2 OBJECTIVE:

The aim of this study is to learn about the machine learning model that can be conveniently developed for each heavy vehicle in a fleet. The model relies on seven predictors: number of stops, stop time, average moving speed, characteristic acceleration, aerodynamic speed squared, change in kinetic energy and change in potential energy. Our proposed model can easily be developed and deployed for each individual vehicle in a fleet in order to optimize fuel consumption over the entire fleet.

1.3 EXISTING SYSTEM

Support Vector Machines (SVM) in predicting vehicle fuel consumption, focusing on its effectiveness in handling complex relationships between input features. SVM is employed to model the relationship between variables such as Mass Air Flow, Vehicle Speed, Revolutions Per Minute, and Throttle Position Sensor, leveraging its ability to find optimal hyperplanes for classification tasks. These systems typically utilize SVM's kernel methods, including polynomial, radial basis function (RBF), and sigmoid kernels, to transform data into higher-dimensional spaces where linear separation of classes is feasible. By maximizing the margin between support vectors and the decision boundary, SVM enhances the accuracy of fuel

consumption predictions.

1.3.1 DRAWBACKS: -

- Especially with large datasets, making it slower to train compared to simpler algorithms.
- Requires significant memory for large datasets, as it needs to store all support vectors.
- SVM does not scale well with large datasets, as the training complexity is highly dependent on the number of support vectors.

CHAPTER: 2

LITERATURE REVIEW

2.1 LITERATURE SURVEY

Title: Monitoring co2 emissions from hdv in Europe-an experimental proof of concept of the proposed methodological approach.

Author: G. Fontaras, R. Luz, K. Anagnostopoulus, Rautela

Year: 2014

Description: The European Commission in joint collaboration with Heavy Duty Vehicle manufactures, the Graz University of Technology and other consulting and research bodies has been preparing a new legislative framework for monitoring and reporting CO2 emissions from Heavy Duty Vehicles (HDVs) in Europe. In contrast to passenger cars and light commercial vehicles, for which monitoring is performed through chassis dyno measurements, and considering the diversity and particular characteristics of the HDV market, it was decided that the core of the proposed methodology should be based on a combination of component testing and vehicle simulation. Emphasis is put on accurately simulating the performance of different vehicle components and achieving realistic fuel consumption results. A proof of concept was launched aiming to test and prove that these targets are achievable. CO2 emissions of the vehicle are then calculated using the aforementioned input data for predefined representative driving cycles and mission profiles. For the two Heavy Duty vehicles tested and simulated on the same test route, fuel consumption was calculated always within a $\pm 3\%$ range from the real world measurement, and in several cases even closer than that (in the order of $\pm 1.5\%$). Given the variability of the actual measurement ($\sigma = 2\%$), it is concluded that a future certification scheme can be based on vehicle simulation tools.

Title: Fuel consumption prediction of fleet vehicles using machine learning: A comparative study,”

Author: S. Wickramanayake and H. D. Bandara,

Year: 2016

Description: Ability to model and predict the fuel consumption is vital in enhancing fuel economy of vehicles and preventing fraudulent activities in fleet management. Fuel consumption of a vehicle depends on several internal factors such as distance, load, vehicle characteristics, and driver behavior, as well as external factors such as road conditions, traffic, and weather. However, not all these factors may be measured or available for the fuel consumption analysis. We consider a case where only a subset of the aforementioned factors is available as a multi-variate time series from a long distance, public bus. Hence, the challenge is to model and/or predict the fuel consumption only with the available data, while still indirectly capturing as much as influences from other internal and external factors. Machine Learning (ML) is suitable in such analysis, as the model can be developed by learning the patterns in data. In this paper, we compare the predictive ability of three ML techniques in predicting the fuel consumption of the bus, given all available parameters as a time series. Based on the analysis, it can be concluded that the random forest technique produces a more accurate prediction compared to both the gradient boosting and neural networks.

Title: “Modeling heavy/medium duty fuel consumption based on drive cycle properties,”.

Author: L. Wang, A. Duran, J. Gonder, and K. Kelly.

Year: 2015

Description: This paper presents multiple methods for predicting heavy/medium-duty vehicle fuel consumption based on driving cycle information. A polynomial model, a black box artificial neural net model, a polynomial neural network model, and a multivariate adaptive regression splines (MARS) model were developed and verified using data collected from chassis testing performed on a parcel delivery diesel truck operating over the Heavy Heavy-Duty Diesel Truck (HHDDT), City Suburban Heavy Vehicle Cycle (CSHVC), New York Composite Cycle (NYCC), and hydraulic hybrid vehicle (HHV) drive cycles. Each model was trained using one of four drive cycles as a training cycle and the other three as testing cycles. By comparing the training and testing results, a representative training cycle was chosen and used to further tune each method. HHDDT as the training cycle gave the best predictive results, because HHDDT contains a variety of drive characteristics, such as high speed, acceleration, idling, and deceleration. Among the four model approaches, MARS gave the best predictive performance, with an average percent error of -1.84% over the four chassis dynamometer drive cycles. To further evaluate the accuracy of the predictive models, the approaches were applied to real-world data. MARS outperformed the other three approaches, providing an average percent error of -2.2% over four real-world road segments. The MARS model performance was then compared to powertrain modeling results over HHDDT, CSHVC, NYCC, and HHV drive cycles using NREL's Future Automotive Systems Technology Simulator (FAST Sim). The results indicated that the MARS method achieved comparable predictive performance with FAST Sim.

Title: “Application of machine learning for fuel consumption modelling of trucks, ”..

Author: F. Perrotta, T. Parry, and L. C. Neves.

Year: 2017

Description: This paper presents the application of three Machine Learning techniques to fuel consumption modelling of articulated trucks for a large dataset. In particular, Support Vector Machine (SVM), Random Forest (RF), and Artificial Neural Network (ANN) models have been developed for the purpose and their performance compared. Fleet managers use telematic data to monitor the performance of their fleets and take decisions regarding maintenance of the vehicles and training of their drivers. The data, which include fuel consumption, are collected by standard sensors (SAE J1939) for modern vehicles. Data regarding the characteristics of the road come from the Highways Agency Pavement Management System (HAPMS) of Highways England, the manager of the strategic road network in the UK. Together, these data can be used to develop a new fuel consumption model, which may help fleet managers in reviewing the existing vehicle routing decisions, based on road geometry. The model would also be useful for road managers to better understand the fuel consumption of road vehicles and the influence of road geometry. Ten-fold cross-validation has been performed to train the SVM, RF, and ANN models. Results of the study shows the feasibility of using telematic data together with the information in HAPMS for the purpose of modelling fuel consumption. The study also shows that although all the three methods make it possible to develop models with good precision, the RF slightly outperforms SVM and ANN giving higher R^2 , and lower error

Title: “Assessing the regeneration potential for a refuse truck over a real-world duty cycle,”

Author: A. Ivanco, R. Johri, and Z. Filipi.

Year: 2012

Description: The majority of a refuse truck collection cycle consists of frequent Stop and Go events while moving from one household to another. The nature of this driving mission creates the opportunity to reduce fuel consumption by capturing and re-using the kinetic energy normally wasted during braking. This paper includes the evaluation of the brake energy available for regeneration from the conventional drivetrain; the description of the impact of the vehicle variable mass and auxiliary loads; a model validation over a real-world duty cycle; and the potential for an increase in fuel efficiency through hybridization of the drivetrain. The Hydraulic Hybrid (HH) technology is selected since it has a large power density. Special attention is paid to two main issues related to refuse truck modeling: i) the fact that each time the trash bin is picked up, the vehicle mass increases in a discrete manner; ii) the load from the additional auxiliary Power Take-Off unit used to lift and compact the refuse. The model of the complex vehicle system is validated with data from real-world refuse collection duty cycles in Texas.

2.2 PROPOSED SYSTEM

Aims to develop a robust fuel consumption prediction model using Logistic Regression, focusing on its simplicity, interpretability, and efficiency. Logistic Regression is chosen for its ability to model binary outcomes effectively, making it suitable for predicting vehicle fuel consumption. Logistic Regression transforms input features into probabilities using a sigmoid function, enabling straightforward interpretation of predictions. The system emphasizes optimizing fuel efficiency and economic benefits by accurately estimating fuel consumption. By leveraging Logistic Regression's strengths in handling linear relationships and providing probabilistic outputs, the proposed system aims to deliver reliable predictions that can assist manufacturers in making informed decisions to improve vehicle performance and reduce operational costs.

2.2.1 PROPOSED SYSTEM ADVANTAGES:

- The results are straightforward to interpret, with output probabilities that can be easily explained.
- Less computational power compared to more complex algorithms.
- Particularly effective for binary classification problems.
-  Logistic Regression outputs probabilities.

CHAPTER 3

PROJECT DESCRIPTION

3.1 GENERAL:

We designed a simple online quiz administered by a custom-built webpage to measure the accuracy of calorie estimation in pictures of food, verify the existence of collective wisdom, and analyze data and find patterns and trends that can be useful in the design of calorie tracking apps.

3.2 METHODOLOGIES

3.2.1 MODULES NAME:

1. Data Collection
2. Pre-Processing
3. Splitting
4. Normalize Data
5. Building Model

3.2.2 MODULES EXPLANATION:

1. Data Collection

Data collection is defined as the procedure of collecting, measuring and analysing accurate insights for research using standard validated techniques. A researcher can evaluate their hypothesis on the basis of collected data. In most cases, data collection is the primary and most important step for research, irrespective of the field of research. The approach of data collection is different for different fields of study, depending on the required information.

2. Pre-Processing

In this section we are doing a little bit of data exploration, checking for null values, object data types and other things we might consider in order to keep our data clean and well structured.

3. Splitting

Data splitting is the act of partitioning available data into. two portions, usually for cross-

validatory purposes. one portion of the data is used to develop a predictive model. and the other to evaluate the model's performance.

Training Data: Used for train the model or given as input to the to the learning model

Testing Data: Used for test the model or given as input to the model for prediction.

4. Normalize Data

Normalization is a technique often applied as part of data preparation for machine learning. The goal of normalization is to change the values of numeric columns in the dataset to use a common scale, without distorting differences in the ranges of values or losing information.

5. Model Building

Building machine learning models that have the ability to generalize well on future data requires thoughtful consideration of the data at hand and of assumptions about various available training algorithms. Ultimate evaluation of a machine learning model's quality requires an appropriate selection and interpretation of assessment criteria.

3.3 TECHNIQUE USED OR ALGORITHM USED

3.3.1 EXISTING TECHNIQUE: -

SVM:

Support Vector Machines (SVM) in predicting vehicle fuel consumption, focusing on its effectiveness in handling complex relationships between input features. SVM is employed to model the relationship between variables such as Mass Air Flow, Vehicle Speed, Revolutions Per Minute, and Throttle Position Sensor, leveraging its ability to find optimal hyperplanes for classification tasks. These systems typically utilize SVM's kernel methods, including polynomial, radial basis function (RBF), and sigmoid kernels, to transform data into higher-dimensional spaces where linear separation of classes is feasible. By maximizing the margin between support vectors and the decision boundary, SVM enhances the accuracy of fuel consumption predictions.

3.3.2 PROPOSED TECHNIQUE USED OR ALGORITHM USED:

Logistic Regression:

fuel consumption prediction model using Logistic Regression, focusing on its simplicity, interpretability, and efficiency. Logistic Regression is chosen for its ability to model binary outcomes effectively, making it suitable for predicting vehicle fuel consumption. Logistic Regression transforms input features into probabilities using a sigmoid function, enabling straightforward interpretation of predictions. The system emphasizes optimizing fuel efficiency and economic benefits by accurately estimating fuel consumption. By leveraging Logistic Regression's strengths in handling linear relationships and providing probabilistic outputs, the proposed system aims to deliver reliable predictions that can assist manufacturers in making informed decisions to improve vehicle performance and reduce operational costs.

CHAPTER 4

REQUIREMENTS ENGINEERING

4.1 GENERAL:

Finally, we introduce semantic sentiment analysis concept which concern about the semantic meaning of the word. It would be classified in to two types contextual semantic and conceptual semantic. Contextual semantic deals with considering about neighboring word. Conceptual semantic depends on outside knowledge such as ontologies and semantic network. We consider conceptual semantic as feature which can be incorporate with Naïve Bayes machine learning method to increase the performance of sentiment analysis.

4.2 HARDWARE REQUIREMENTS

The hardware requirements may serve as the basis for a contract for the implementation of the system and should therefore be a complete and consistent specification of the whole system. They are used by software engineers as the starting point for the system design. It should state what the system does and not how it should be implemented.

- PROCESSOR : DUAL CORE 2 DUOS..
- RAM : 4GB DD RAM
- HARD DISK : 250 GB

4.3 SOFTWARE REQUIREMENTS

The software requirements document is the specification of the system. It should include both a definition and a specification of requirements. It is a set of what the system should do rather than how it should do it. The software requirements provide a basis for creating the software requirements specification. It is useful in estimating cost, planning team activities, performing tasks and tracking the teams and tracking the team's progress throughout the development activity.

- Operating System : Windows 7/8/10
- Platform : Vs Code/ Spyder3
- Programming Language : Python
- Front End : HTML, CSS

4.4 FUNCTIONAL REQUIREMENTS

A functional requirement defines a function of a software-system or its component. A function is described as a set of inputs, the behavior, Firstly, the system is the first that achieves the standard notion of semantic security for data confidentiality in attribute-based deduplication systems by resorting to the hybrid cloud architecture.

4.5 NON-FUNCTIONAL REQUIREMENTS

The major non-functional Requirements of the system are as follows:

EFFICIENCY

Our multi-modal event tracking and evolution framework is suitable for multimedia documents from various social media platforms, which can not only effectively capture their multi-modal topics, but also obtain the evolutionary trends of social events and generate effective event summary details over time. Our proposed mmETM model can exploit the multi-modal property of social event, which can effectively model social media documents including long text with related images and learn the correlations between textual and visual modalities to separate the visual-representative topics and non-visual-representative topics.

TECHNIQUE OR ALGORITHM USED:

Logistic Regression is a statistical method used for binary classification, meaning it helps predict whether something belongs to one of two categories. Despite its name, Logistic Regression is actually used for classification tasks rather than regression. The process involves taking multiple input features, such as Mass Air Flow, Vehicle Speed, Revolutions Per Minute, and Throttle Position Sensor, and combining them into a single value using a linear equation. This value is then transformed using a sigmoid function, which converts it into a probability

between 0 and 1. If this probability is greater than or equal to a certain threshold (usually 0.5), the model predicts one category; if it is less, it predicts the other. Logistic Regression is popular because it is simple to implement, easy to interpret, and efficient, making it a good choice for problems where the goal is to classify data into two distinct groups.

CHAPTER 5

DESIGN ENGINEERING

5.1 GENERAL:

Design Engineering deals with the various UML [Unified Modeling language] diagrams for the implementation of project. Design is a meaningful engineering representation of a thing that is to be built. Software design is a process through which the requirements are translated into representation of the software. Design is the place where quality is rendered in software engineering. Design is the means to accurately translate customer requirements into finished product.

5.2 UML DIAGRAMS

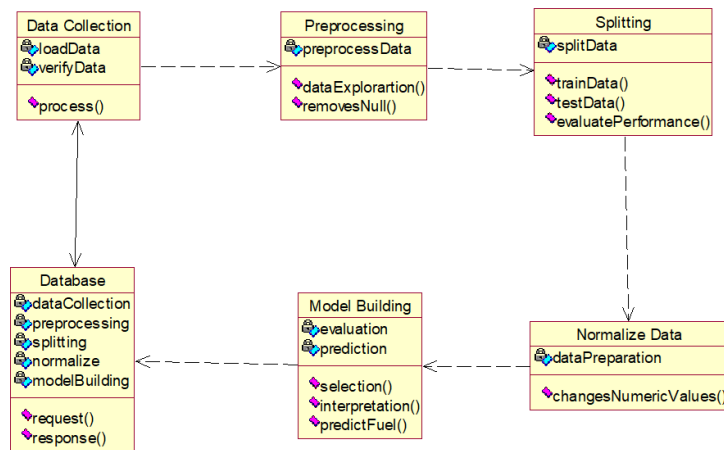
5.2.1 USE CASE DIAGRAM:



EXPLANATION:

The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted. The above diagram consists of user as actor. Each will play a certain role to achieve the concept

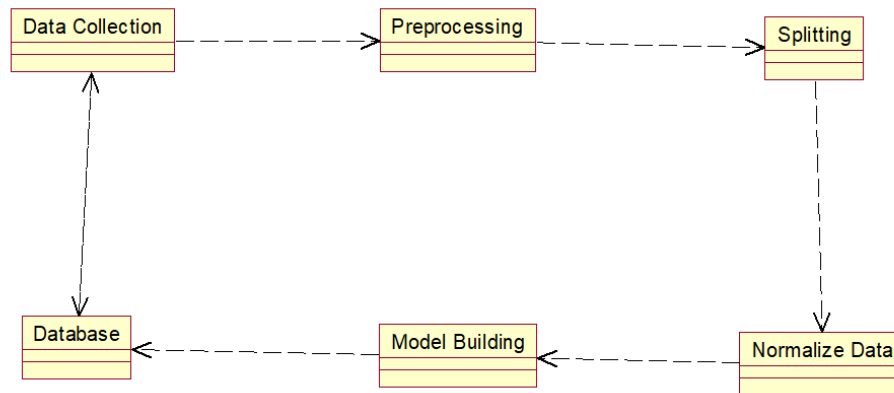
5.2.2 CLASS DIAGRAM:



EXPLANATION:

In this class diagram represents how the classes with attributes and methods are linked together to perform the verification with security. From the above diagram shown the various classes involved in our project.

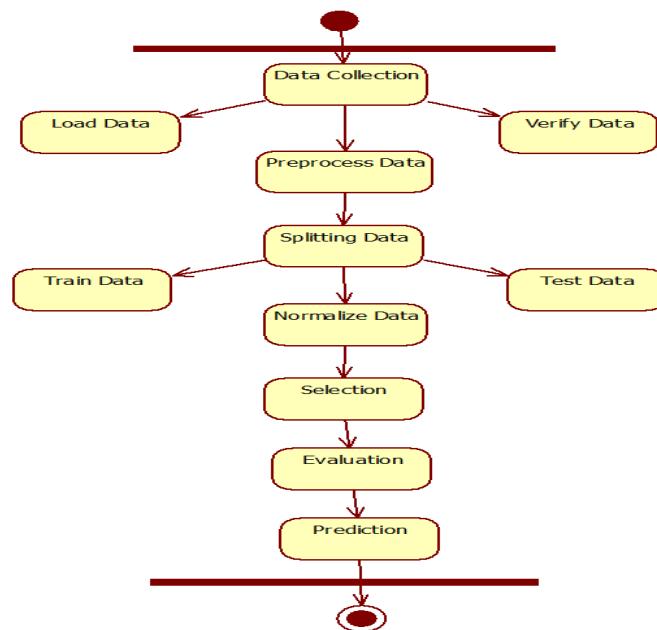
5.2.2 OBJECT DIAGRAM:



EXPLANATION:

In the above diagram tells about the flow of objects between the classes. It is a diagram that shows a complete or partial view of the structure of a modeled system. In this object diagram represents how the classes with attributes and methods are linked together to perform the verification with security

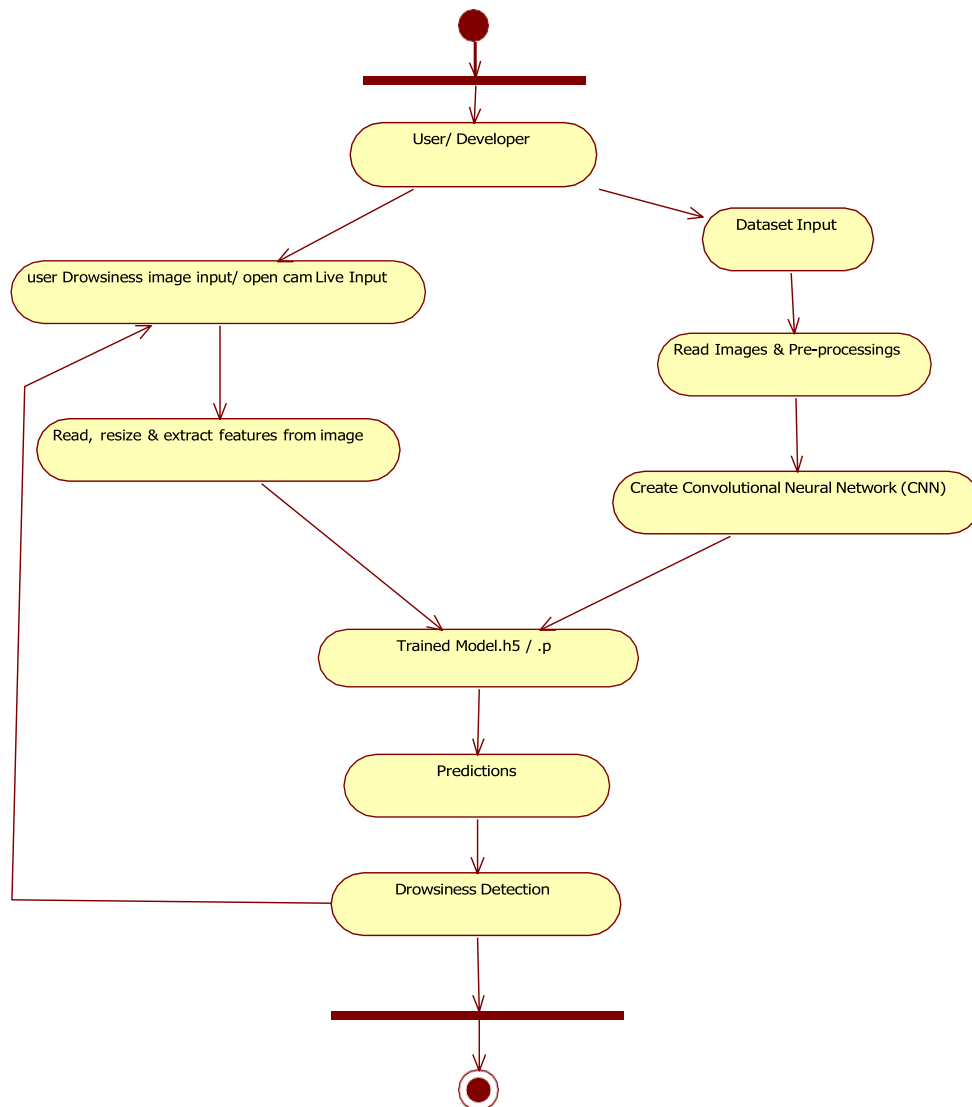
5.2.4 STATE DIAGRAM:



EXPLANATION:

State diagrams are a loosely defined diagram to show workflows of stepwise activities and actions, with support for choice, iteration and concurrency. State diagrams require that the system described is composed of a finite number of states; sometimes, this is indeed the case, while at other times this is a reasonable abstraction. Many forms of state diagrams exist, which differ slightly and have different semantics. It outlines a sequential data processing pipeline, potentially applicable to various domains like Machine Learning or Business processes.

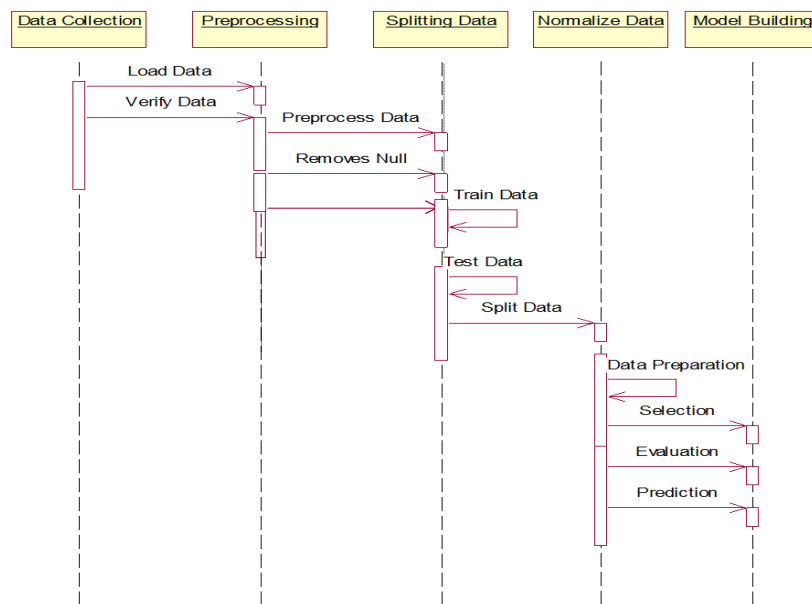
5.2.5 ACTIVITY DIAGRAM:



EXPLANATION:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

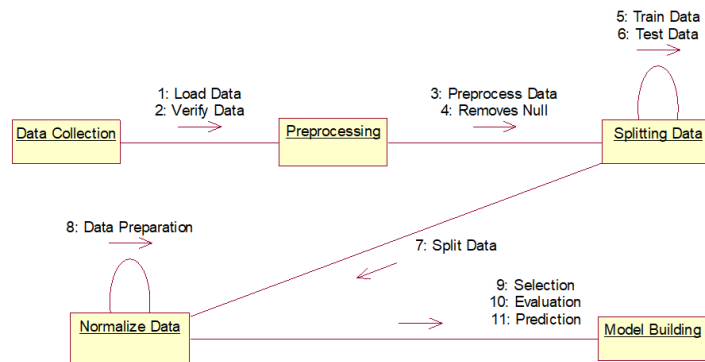
5.2.6 SEQUENCE DIAGRAM:



EXPLANATION:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

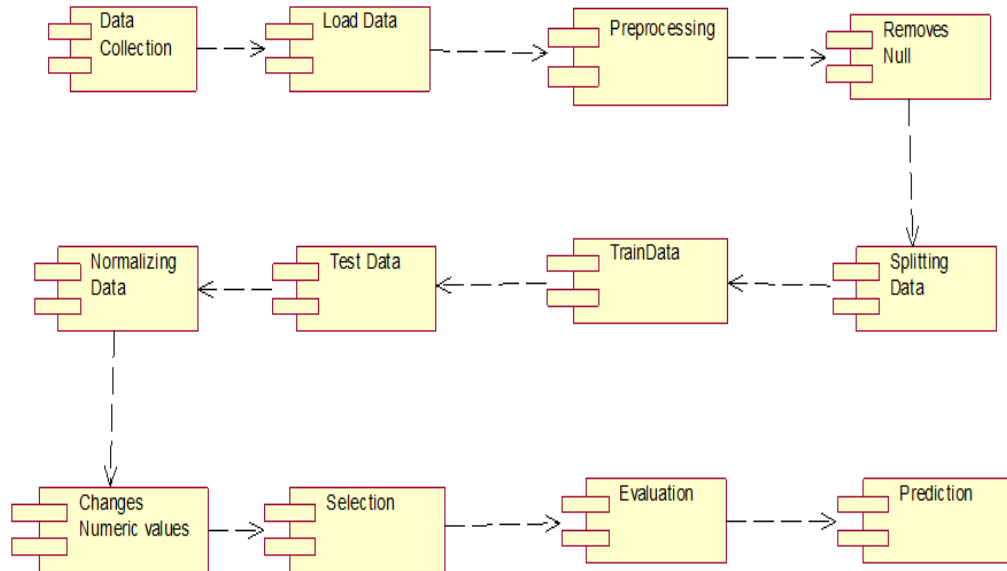
5.2.7 COLLABORATION DIAGRAM:



EXPLANATION:

A collaboration diagram, also called a communication diagram or interaction diagram, is an illustration of the relationships and interactions among software objects in the Unified Modeling Language (UML). The concept is more than a decade old although it has been refined as modeling

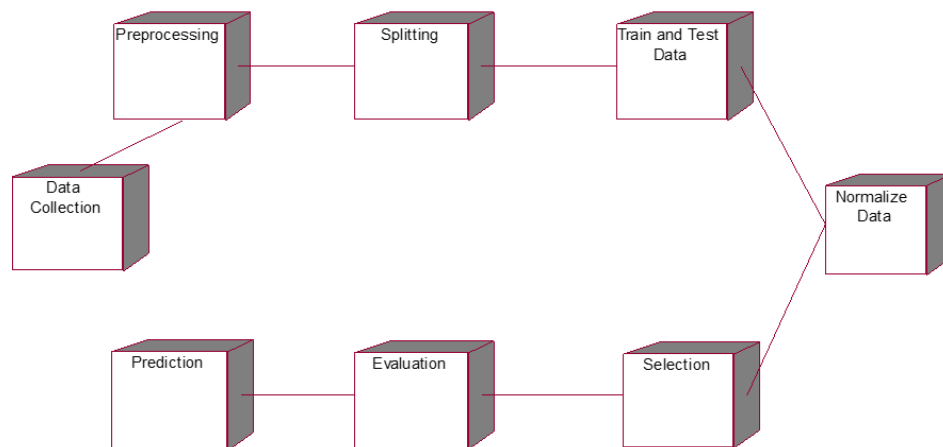
5.2.8 COMPONENT DIAGRAM:



EXPLANATION:

In the Unified Modeling Language, a component diagram depicts how components are wired together to form larger components and or software systems. They are used to illustrate the structure of arbitrarily complex systems. User gives main query and it converted into sub queries and sends through data dissemination to data aggregators. Results are to be showed to user by data aggregators. All boxes are components and arrow indicates dependencies. The provided component diagram outlines data processing potentially applicable to various domains like Machine Learning or business processing.

5.2.9 DEPLOYMENT DIAGRAM:



EXPLANATION:

Deployment Diagram is a type of diagram that specifies the physical hardware on which the software system will execute. It also determines how the software is deployed on the underlying hardware. It maps software pieces of a system to the device that are going to execute it.

5.2.10 SYSTEM ARCHITECTURE:

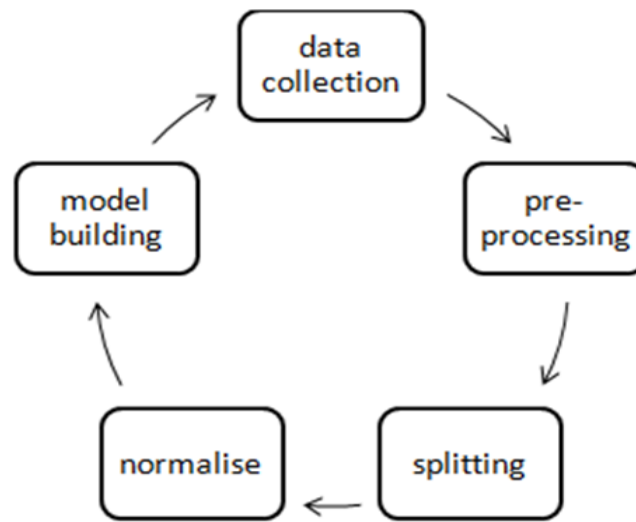


Fig:- proposed model

CHAPTER 6

DEVELOPMENT TOOLS

6.1 Python

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

6.2 History of Python

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.

Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

6.3 Importance of Python

- **Python is Interpreted** – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive** – You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented** – Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language** – Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple

text processing to WWW browsers to games.

6.4 Features of Python

- **Easy-to-learn** – Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read** – Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain** – Python's source code is fairly easy-to-maintain.
- **A broad standard library** – Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode** – Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable** – Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable** – You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases** – Python provides interfaces to all major commercial databases.
- **GUI Programming** – Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable** – Python provides a better structure and support for large programs than shell scripting.

Apart from the above-mentioned features, Python has a big list of good features, few are listed below –

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.

- IT supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.

6.5 Libraries used in python

- numpy - mainly useful for its N-dimensional array objects.
- pandas - Python data analysis library, including structures such as dataframes.
- matplotlib - 2D plotting library producing publication quality figures.
- scikit-learn - the machine learning algorithms used for data analysis and data mining tasks.



Fig: NumPy, Pandas, Matplotlib, Scikit-learn

CHAPTER 7

IMPLEMENTATION

7.1 IMPLEMENTATION

Code:

```
from flask import Flask, request, jsonify, render_template, url_for
import pickle
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

df = pd.read_csv("Data/MY2019 Fuel Consumption Ratings.csv")

# Label Encoding
le_make = LabelEncoder()
df['Make'] = le_make.fit_transform(df['Make'])

le_fuel_type = LabelEncoder()
df['Fuel_Type'] = le_fuel_type.fit_transform(df['Fuel_Type'])

# Converting Pandas DataFrame into Numpy array
X = df[['Engine_Size (L)', 'Make', 'Fuel_Type', 'Cylinders', 'Fuel_Consumption_City',
'Fuel_Consumption_Hwy']].values
y = df[['CO2_Emissions']].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size= 0.25, random_state= 0)

app = Flask(__name__)
model = pickle.load(open('model.pkl', 'rb'))
```

```
@app.route('/')
def home():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    engine = request.form['engine']
    make = request.form['make']
    fuel = request.form['fuel']
    cylinders = request.form['cylinders']
    Fuel_Consumption_City = request.form['Fuel_Consumption_City']
    Fuel_Consumption_Hwy = request.form['Fuel_Consumption_Hwy']

    x1 = [engine, make, fuel, cylinders, Fuel_Consumption_City, Fuel_Consumption_Hwy]
    df = pd.DataFrame(data=[x1], columns=['Engine_Size (L)', 'Make', 'Fuel_Type',
    'Cylinders', 'Fuel_Consumption_City', 'Fuel_Consumption_Hwy'])

    df['Make'] = le_make.transform(df['Make'])
    df['Fuel_Type'] = le_fuel_type.transform(df['Fuel_Type'])

    X = df.iloc[:, :6].values
    ans = model.predict(X)
    output = ans

    return render_template('index.html', prediction_text='Fuel Level: {}'.format(output))

if __name__ == '__main__':
    app.run(debug=False)
```

Python code for regression model:
import matplotlib.pyplot as plt

```

import pandas as pd
import numpy as np
import seaborn as sns
df = pd.read_csv("Data/MY2019 Fuel Consumption Ratings.csv", na_values=['??', '##', '@@',
'&&'])
# Does not restrict the maximum column or filter it to fit the size of the display
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', None)
df.head(30)
df.tail(25)
df['Make'].value_counts()
df['Transmission'].value_counts()
df['Vehicle_Class'].value_counts()
df['Cylinders'].value_counts()
df.describe()
# Data types in the data set
df.dtypes
for column in df.columns:
    print(column, ", n(missing values) = ", df[column].isnull().values.sum()*100/len(df), ",
unique values = ", df[column].nunique())
    print("")
# Another method
# If True, then NULL value is present and vice-versa
df.isnull()
# Sum the no. of NULL value will show the no of NULL values in the dataset
df.isnull().sum()
cdf = df[['Engine_Size (L)', 'Cylinders', 'Fuel_Consumption_City', 'Fuel_Consumption_Hwy',
'Fuel_Consumption_Comb', 'CO2_Emissions', 'CO2_Rating', 'Smog_Rating']]
cdf.head(10)
correlation_matrix = cdf.corr()
correlation_matrix

```

```
corr = correlation_matrix.iloc[:6, :6]
corr.style.background_gradient(cmap='coolwarm')
plt.scatter(cdf['Engine_Size (L)'], cdf['CO2_Emissions'], color='blue')
plt.xlabel("Engine Size (L)")
plt.ylabel("CO2_Emission")
plt.show()
plt.scatter(cdf['Engine_Size (L)'], cdf['CO2_Rating'], color='blue')
plt.xlabel("Engine Size (L)")
plt.ylabel("CO2_Rating")
plt.show()
plt.scatter(cdf['Engine_Size (L)'], cdf['Smog_Rating'], color='blue')
plt.xlabel("Engine Size (L)")
plt.ylabel("Smog_Rating")
plt.show()
plt.scatter(cdf['Engine_Size (L)'], cdf['Smog_Rating'], color='blue')
plt.xlabel("Engine Size (L)")
plt.ylabel("Smog_Rating")
plt.show()
plt.scatter(cdf['Cylinders'], cdf['CO2_Rating'], color='red')
plt.xlabel("Cylinders")
plt.ylabel("CO2_Rating")
plt.show()
plt.scatter(cdf['Cylinders'], cdf['Smog_Rating'], color='red')
plt.xlabel("Cylinders")
plt.ylabel("Smog Rating")
plt.show()
plt.scatter(cdf['Fuel_Consumption_Comb'], cdf['CO2_Emissions'], color='yellow')
plt.xlabel("Fuel_Consumption_Comb")
plt.ylabel("CO2_Emission")
plt.show()
plt.scatter(cdf['Fuel_Consumption_Comb'], cdf['CO2_Rating'], color='yellow')
```

```
plt.xlabel("Fuel_Consumption_Comb")
plt.ylabel("CO2_Rating")
plt.show()
plt.scatter(cdf['Fuel_Consumption_Comb'], cdf['Smog_Rating'], color='yellow')
plt.xlabel("Fuel_Consumption_Comb")
plt.ylabel("Smog_Rating")
plt.show()
plt.scatter(cdf['Fuel_Consumption_City'], cdf['CO2_Emissions'], color='green')
plt.xlabel("Fuel Consumption in City")
plt.ylabel("CO2_Emission")
plt.show()
plt.scatter(cdf['Fuel_Consumption_City'], cdf['CO2_Rating'], color='green')
plt.xlabel("Fuel Consumption in City")
plt.ylabel("CO2_Rating")
plt.show()
plt.scatter(cdf['Fuel_Consumption_City'], cdf['Smog_Rating'], color='green')
plt.xlabel("Fuel Consumption in City")
plt.ylabel("Smog_Rating")
plt.show()
plt.scatter(cdf['Fuel_Consumption_Hwy'], cdf['CO2_Emissions'], color='orange')
plt.xlabel("Fuel_Consumption_Hwy")
plt.ylabel("Co2_Emission")
plt.show()
plt.scatter(cdf['Fuel_Consumption_Hwy'], cdf['CO2_Rating'], color='orange')
plt.xlabel("Fuel_Consumption_Hwy")
plt.ylabel("Co2_Rating")
plt.show()
plt.scatter(cdf['Fuel_Consumption_Hwy'], cdf['Smog_Rating'], color='orange')
plt.xlabel("Fuel_Consumption_Hwy")
plt.ylabel("Smog_Rating")
plt.show()
```

```

X = cdf[['Engine_Size (L)']].values
X[0:5]
y = cdf[['CO2_Emissions']] .values
y[0:5]
from sklearn.model_selection import train_test_split, cross_val_score
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)
lst_id = []
for i in range(1, len(cdf)+1):
    lst_id.append(i)
print(lst_id)
plt.scatter(cdf['Engine_Size (L)'], cdf['CO2_Emissions'], color='blue')
plt.xlabel("Engine Size (L)")
plt.ylabel("C02 Emissions")
plt.show()
from sklearn.linear_model import LinearRegression
linear_reg = LinearRegression(fit_intercept=True, copy_X=True, n_jobs=-1)
# n_jobs assigns no. of CPU cores to be utilized for Model rendering
linear_reg.fit(X_train, y_train)
print("Slope (Coeffients)= ", linear_reg.coef_)
print("Intercept= ", linear_reg.intercept_)
# If cv=5, it cross validates 5 times. So if cv=n, it'll cross validate n no. of times
linear_reg_cv_score = cross_val_score(linear_reg, X_train, y_train, cv=5)
print("Linear Regression cross validated score= ", linear_reg_cv_score)
# So it's beneficial to take the mean of the cross validation score
linear_reg_cv_score = np.mean(cross_val_score(linear_reg, X_train, y_train, cv=5))
print("Linear Regression cross validated score= ", linear_reg_cv_score)

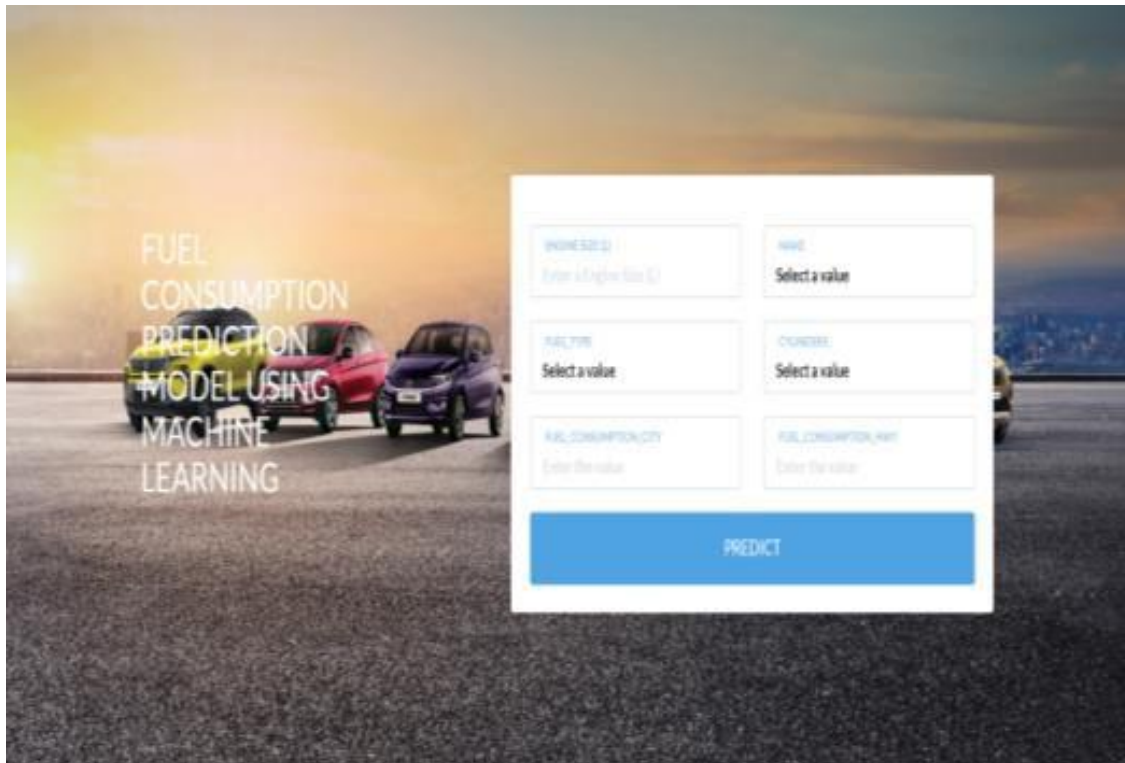
```

SNAPSHOTS

7.1 GENERAL:

This project implements like web application using COREJAVA and the Server process is maintained using the SOCKET & SERVERSOCKET and the Design part is played by Cascading Style Sheet.

7.2 SNAPSHOTS





The image shows a web application interface for fuel consumption prediction. The background features a sunset over a road with three cars (yellow, red, and purple). The text "FUEL CONSUMPTION PREDICTION MODEL USING MACHINE LEARNING" is overlaid on the left. On the right, there is a form with six input fields and a "PREDICT" button. The inputs are: ENGINE SIZE (23), MAKE (Ford), FUEL TYPE (D), CYLINDERS (4), FUEL CONSUMPTION (CITY) (14), and FUEL CONSUMPTION (HWH) (18).

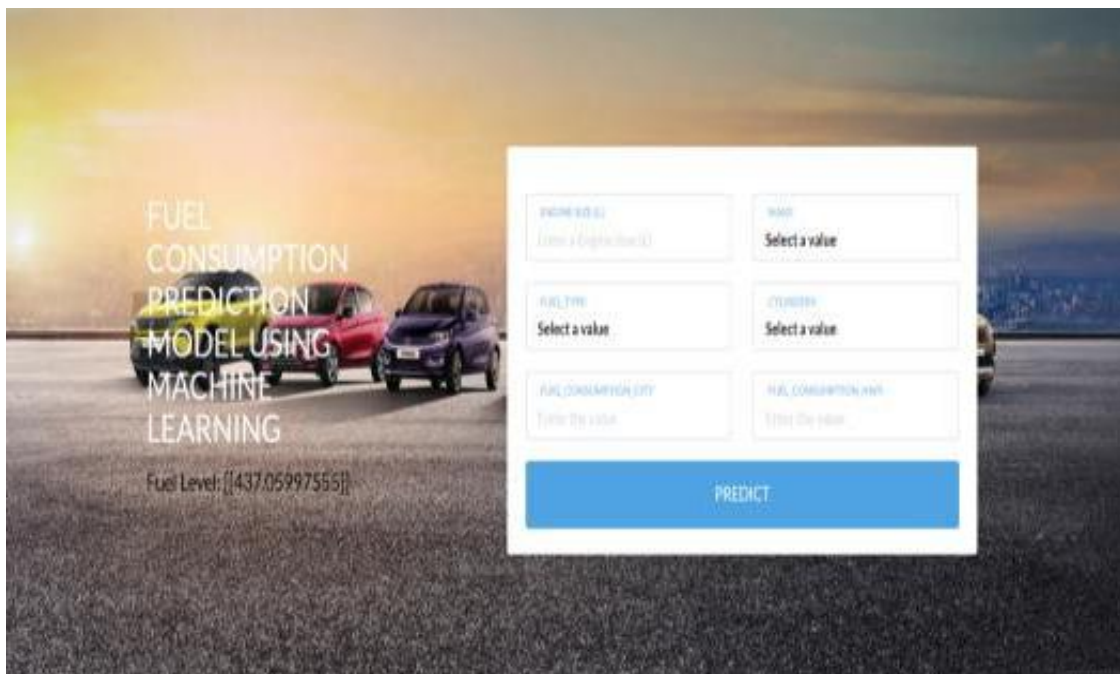
ENGINE SIZE	MAKE
23	Ford

FUEL TYPE	CYLINDERS
D	4

FUEL CONSUMPTION (CITY)	FUEL CONSUMPTION (HWH)
14	18

PREDICT

Fig:7.2.1 Sample Input



The image shows the same web application interface as Fig:7.2.1, but with the output displayed. The input fields are now disabled and show "Select a value" or "Enter a value". The output "Fuel Level: [[437.05997555]]" is displayed below the text "FUEL CONSUMPTION PREDICTION MODEL USING MACHINE LEARNING". The "PREDICT" button is still present.

ENGINE SIZE	MAKE
Enter a value	Select a value

FUEL TYPE	CYLINDERS
Select a value	Select a value

FUEL CONSUMPTION (CITY)	FUEL CONSUMPTION (HWH)
Enter the value	Enter the value

Fuel Level: [[437.05997555]]

PREDICT

Fig:7.2.2: Detected Output

CHAPTER 8

SOFTWARE TESTING

8.1 GENERAL:

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

8.2 DEVELOPING METHODOLOGIES

The test process is initiated by developing a comprehensive plan to test the general functionality and special features on a variety of platform combinations. Strict quality control procedures are used. The process verifies that the application meets the requirements specified in the system requirements document and is bug free. The following are the considerations used to develop the framework from developing the testing methodologies.

8.3 TYPES OF TESTS

8.3.1 Unit testing:

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program input produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

8.3.2 Functional test:

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals. Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures: interfacing systems or procedures must be invoked.

8.3.3 System Test:

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

8.3.4 Performance Test:

The Performance test ensures that the output be produced within the time limits, and the time taken by the system for compiling, giving response to the users and request being send to the system for to retrieve the results.

8.3.5 Integration Testing:

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level interact without error.

8.3.6 Acceptance Testing:

User Acceptance Testing is a critical phase of any project and requires significant

participation by the end user. It also ensures that the system meets the functional requirements.

Acceptance testing for Data Synchronization:

- The Acknowledgements will be received by the Sender Node after the Packets are received by the Destination Node
- The Route add operation is done only when there is a Route request in need
- The Status of Nodes information is done automatically in the Cache Updation process

8.3.7 Build the test plan:

Any project can be divided into units that can be further performed for detailed processing. Then a testing strategy for each of this unit is carried out. Unit testing helps to identify the possible bugs in the individual component, so the component that has bugs can be identified and can be rectified from error.

CHAPTER 9

FUTURE ENHANCEMENT

9.1 FUTURE ENHANCEMENTS:

Future work also includes investigating the minimum distance required for training each model and analyzing how often does a model need to be synchronized with the physical system in operation by using online training in order to maintain the prediction accuracy of the model.

CHAPTER 10

CONCLUSION

10.1 CONCLUSION:

This project presented a machine learning model that can be conveniently developed for each heavy vehicle in a fleet. The model relies on seven predictors: number of stops, stop time, average moving speed, characteristic acceleration, aerodynamic speed squared, change in kinetic energy and change in potential energy. The last two predictors are introduced in this paper to help capture the average dynamic behavior of the vehicle. All of the predictors of the model are derived from vehicle speed and road grade. These variables are readily available from telematics devices that are becoming an integral part of connected vehicles. Moreover, the predictors can be easily computed on-board from these two variables. The model predictors are aggregated over a fixed distance traveled (i.e., window) instead of a fixed time interval. This mapping of the input space to the distance domain aligns with the domain of the target output, and produced a machine learning model for fuel consumption with an RMSE < 0.015 l/100km. Different model configurations with 1, 2, and 5 km window sizes were evaluated. The results show that the 1 km window has the highest accuracy. This model is able to predict the actual fuel consumption on a per 1 km-basis with a CD of 0.91. This performance is closer to that of physics-based models and the proposed model improves upon previous machine learning models that show comparable results only for entire long-distance trips. Selecting an adequate window size should take into consideration the cost of the model in terms of data collection and on-board computation. Moreover, the window size is likely to be application-dependent. For fleets with short trips (e.g., construction vehicles within a site) or urban traffic routes, a 1 km window size is recommended. For long-haul fleets, a 5 km window size may be sufficient. In this study, the duty cycles consisted of both highway and city traffic and therefore, the 1 km window was more adequate than the 5 km window. Future work includes understanding these differentiating factors and the selection of the appropriate window size.

REFERENCES

- [1] B. Lee, L. Quinones, and J. Sanchez, “Development of greenhouse gas emissions model for 2014-2017 heavy-and medium-duty vehicle compliance,” SAE Technical Paper, Tech. Rep., 2011.
- [2] G. Fontaras, R. Luz, K. Anagnostopoulus, D. Savvidis, S. Hausberger, and M. Rexeis, “Monitoring co2 emissions from hdv in europe-an experimental proof of concept of the proposed methodolgical approach,” in 20th International Transport and Air Pollution Conference, 2014.
- [3] S. Wickramanayake and H. D. Bandara, “Fuel consumption prediction of fleet vehicles using machine learning: A comparative study,” in Moratuwa Engineering Research Conference (MERCon), 2016. IEEE, 2016, pp. 90–95.
- [4] L. Wang, A. Duran, J. Gonder, and K. Kelly, “Modeling heavy/mediumduty fuel consumption based on drive cycle properties,” SAE Technical Paper, Tech. Rep., 2015.
- [5] Fuel Economy and Greenhouse gas exhaust emissions of motor vehicles Subpart B - Fuel Economy and Carbon-Related Exhaust Emission Test Procedures, Code of Federal Regulations Std. 600.111-08, Apr 2014.
- [6] SAE International Surface Vehicle Recommended Practice, Fuel Consumption Test Procedure - Type II, Society of Automotive Engineers Std., 2012.
- [7] F. Perrotta, T. Parry, and L. C. Neves, “Application of machine learning for fuel consumption modelling of trucks,” in Big Data (Big Data), 2017 IEEE International Conference on. IEEE, 2017, pp. 3810–3815.
- [8] S. F. Haggis, T. A. Hansen, K. D. Hicks, R. G. Richards, and R. Marx, “In-use evaluation of fuel economy and emissions from coal haul trucks using modified sae j1321 procedures and pems,” SAE International Journal of Commercial Vehicles, vol. 1, no. 2008-01-1302, pp. 210–221, 2008.
- [9] A. Ivanco, R. Johri, and Z. Filipi, “Assessing the regeneration potential for a refuse truck over a real-world duty cycle,” SAE International Journal of Commercial Vehicles, vol. 5, no. 2012-01-1030, pp. 364–370, 2012.
- [10] A. A. Zaidi, B. Kulcsr, and H. Wymeersch, “Back-pressure traffic signal control with fixed and adaptive routing for urban vehicular networks,” IEEE Transactions on Intelligent Transportation Systems, vol. 17, no. 8, pp. 2134–2143, Aug 2016.

- [11] J. Zhao, W. Li, J. Wang, and X. Ban, “Dynamic traffic signal timing optimization strategy incorporating various vehicle fuel consumption characteristics,” *IEEE Transactions on Vehicular Technology*, vol. 65, no. 6, pp. 3874–3887, June 2016.
- [12] G. Ma, M. Ghasemi, and X. Song, “Integrated powertrain energy management and vehicle coordination for multiple connected hybrid electric vehicles,” *IEEE Transactions on Vehicular Technology*, vol. 67, no. 4, pp. 2893–2899, April 2018.
- [13] A. C. Brandt, E. A. Frame, and R. W. Warden, “Sae j1321 testing using m1083a1 fmvts,” *SOUTHWEST RESEARCH INST SAN ANTONIO TX TARDEC FUELS AND LUBRICANTS RESEARCH FACILITY*, Tech. Rep., 2010.
- [14] W. Zeng, T. Miwa, and T. Morikawa, “Exploring trip fuel consumption by machine learning from gps and can bus data,” *Journal of the Eastern Asia Society for Transportation Studies*, vol. 11, pp. 906–921, 2015.